

Fraud Detection - Methodology Notebook

Arjun Sekhar

2026-04-14

1. Introduction

1.1. Objective

This notebook documents the analytical workflow used to investigate fraud detection using both traditional classification models and a survival-based approach. The aim is to compare the baseline banking-oriented models such as the Logistic Regression against a Random Survival Forest (RSF) framework under extreme class imbalance.

1.2. Research Motivation

Fraud detection is traditionally handled as a binary classification problem. However, fraud may also be understood using a time-to-event approach, where the question is not to detect whether fraud occurs, but also with how the fraud risk evolves over time.

1.3. Analytical Roadmap

This notebook is structured to move from data understanding through to model development, evaluation, and interpretation, with a focus on both performance and explainability. It is thus organised as follows:

1. Data Description: An overview of the dataset and its key characteristics.
2. Data Preparation: Construction of modelling variables, including the proxy time index and handling of class imbalance.
3. Exploratory Data Analysis (EDA): Examination of feature distributions and class imbalance patterns.
4. Modelling Framework: Implementation of four model configurations, including Logistic Regression and Random Survival Forest (RSF), with and without SMOTE.
5. Results: Evaluation of model performance using sensitivity, AUC, and AUPRC across repeated train-test splits.
6. Explainability: Interpretation of model behaviour using global and local explanation techniques, including SHAP and LIME.
7. Discussion and Banking Implications: Analysis of model performance and relevance to real-world fraud detection workflows.
8. Limitations and Conclusion: Summary of findings, methodological limitations, and directions for future work.

2. Libraries

Some of the libraries that we use in this process that are to be loaded here include:

```
library(tidyverse)
library(caret)
```

```
library(pROC)
library(PRRROC)
library(randomForest)
library(randomForestSRC)
library(survival)
library(survminer)
```

Additionally to handle the Explainability, we incorporate the following packages specific to this:

```
library(SHAPforxgboost)
library(iml)
library(lime)
```

3. Data Description

3.1. Load Data

To start things off, we will set the seed, ingest the dataset and then omit the missing values. This will make the dataset become analysis-ready.

```
## Step 1: Set seed
set.seed(44940076)

## Step 3: Load Data
fraud_data <- read.csv('creditcard.csv', header = TRUE)

## Step 4: Remove Missing Values
fraud_data <- na.omit(fraud_data)
```

3.2. Missing Values Check

We will quickly check whether any of the columns have missing values in them.

```
colSums(is.na(fraud_data))
```

```
##   Time     V1     V2     V3     V4     V5     V6     V7     V8     V9     V10
##      0      0      0      0      0      0      0      0      0      0      0
##   V11    V12    V13    V14    V15    V16    V17    V18    V19    V20    V21
##      0      0      0      0      0      0      0      0      0      0      0
##   V22    V23    V24    V25    V26    V27    V28 Amount  Class
##      0      0      0      0      0      0      0      0      0      0
```

Good, this means that there is nothing much to worry about from the missingness standpoint.

3.3. Structure and Dimensions

This next step will involve understanding the structure and dimensions, such that we are able to understand the specifics of the data we are dealing with.

```
## [1] 284807      31

## Rows: 284,807
## Columns: 31
## $ Time      <dbl> 0, 0, 1, 1, 2, 2, 4, 7, 7, 9, 10, 10, 10, 11, 12, 12, 12, 13, 1~
## $ V1        <dbl> -1.3598071, 1.1918571, -1.3583541, -0.9662717, -1.1582331, -0.4~
## $ V2        <dbl> -0.07278117, 0.26615071, -1.34016307, -0.18522601, 0.87773675, ~
## $ V3        <dbl> 2.53634674, 0.16648011, 1.77320934, 1.79299334, 1.54871785, 1.1~
## $ V4        <dbl> 1.37815522, 0.44815408, 0.37977959, -0.86329128, 0.40303393, -0~
## $ V5        <dbl> -0.33832077, 0.06001765, -0.50319813, -0.01030888, -0.40719338, ~
## $ V6        <dbl> 0.46238778, -0.08236081, 1.80049938, 1.24720317, 0.09592146, -0~
## $ V7        <dbl> 0.239598554, -0.078802983, 0.791460956, 0.237608940, 0.59294074~
## $ V8        <dbl> 0.098697901, 0.085101655, 0.247675787, 0.377435875, -0.27053267~
## $ V9        <dbl> 0.3637870, -0.2554251, -1.5146543, -1.3870241, 0.8177393, -0.56~
## $ V10       <dbl> 0.09079417, -0.16697441, 0.20764287, -0.05495192, 0.75307443, --
## $ V11       <dbl> -0.55159953, 1.61272666, 0.62450146, -0.22648726, -0.82284288, ~
## $ V12       <dbl> -0.61780086, 1.06523531, 0.06608369, 0.17822823, 0.53819555, 0.~
## $ V13       <dbl> -0.99138985, 0.48909502, 0.71729273, 0.50775687, 1.34585159, -0~
## $ V14       <dbl> -0.31116935, -0.14377230, -0.16594592, -0.28792375, -1.11966983~
## $ V15       <dbl> 1.468176972, 0.635558093, 2.345864949, -0.631418118, 0.17512113~
## $ V16       <dbl> -0.47040053, 0.46391704, -2.89008319, -1.05964725, -0.45144918,~
## $ V17       <dbl> 0.207971242, -0.114804663, 1.109969379, -0.684092786, -0.237033~
```

```
## $ V18 <dbl> 0.02579058, -0.18336127, -0.12135931, 1.96577500, -0.03819479, ~
## $ V19 <dbl> 0.40399296, -0.14578304, -2.26185710, -1.23262197, 0.80348692, ~
## $ V20 <dbl> 0.25141210, -0.06908314, 0.52497973, -0.20803778, 0.40854236, 0~
## $ V21 <dbl> -0.018306778, -0.225775248, 0.247998153, -0.108300452, -0.00943~
## $ V22 <dbl> 0.277837576, -0.638671953, 0.771679402, 0.005273597, 0.79827849~
## $ V23 <dbl> -0.110473910, 0.101288021, 0.909412262, -0.190320519, -0.137458~
## $ V24 <dbl> 0.06692807, -0.33984648, -0.68928096, -1.17557533, 0.14126698, ~
## $ V25 <dbl> 0.12853936, 0.16717040, -0.32764183, 0.64737603, -0.20600959, --
## $ V26 <dbl> -0.18911484, 0.12589453, -0.13909657, -0.22192884, 0.50229222, ~
## $ V27 <dbl> 0.133558377, -0.008983099, -0.055352794, 0.062722849, 0.2194222~
## $ V28 <dbl> -0.021053053, 0.014724169, -0.059751841, 0.061457629, 0.2151531~
## $ Amount <dbl> 149.62, 2.69, 378.66, 123.50, 69.99, 3.67, 4.99, 40.80, 93.20, ~
## $ Class <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
```

##	Time	V1	V2	V3
##	Min. : 0	Min. : -56.40751	Min. : -72.71573	Min. : -48.3256
##	1st Qu.: 54202	1st Qu.: -0.92037	1st Qu.: -0.59855	1st Qu.: -0.8904
##	Median : 84692	Median : 0.01811	Median : 0.06549	Median : 0.1798
##	Mean : 94814	Mean : 0.00000	Mean : 0.00000	Mean : 0.0000
##	3rd Qu.: 139321	3rd Qu.: 1.31564	3rd Qu.: 0.80372	3rd Qu.: 1.0272
##	Max. : 172792	Max. : 2.45493	Max. : 22.05773	Max. : 9.3826
##	V4	V5	V6	V7
##	Min. : -5.68317	Min. : -113.74331	Min. : -26.1605	Min. : -43.5572
##	1st Qu.: -0.84864	1st Qu.: -0.69160	1st Qu.: -0.7683	1st Qu.: -0.5541
##	Median : -0.01985	Median : -0.05434	Median : -0.2742	Median : 0.0401
##	Mean : 0.00000	Mean : 0.00000	Mean : 0.0000	Mean : 0.0000
##	3rd Qu.: 0.74334	3rd Qu.: 0.61193	3rd Qu.: 0.3986	3rd Qu.: 0.5704
##	Max. : 16.87534	Max. : 34.80167	Max. : 73.3016	Max. : 120.5895
##	V8	V9	V10	V11
##	Min. : -73.21672	Min. : -13.43407	Min. : -24.58826	Min. : -4.79747
##	1st Qu.: -0.20863	1st Qu.: -0.64310	1st Qu.: -0.53543	1st Qu.: -0.76249
##	Median : 0.02236	Median : -0.05143	Median : -0.09292	Median : -0.03276
##	Mean : 0.00000	Mean : 0.00000	Mean : 0.00000	Mean : 0.00000
##	3rd Qu.: 0.32735	3rd Qu.: 0.59714	3rd Qu.: 0.45392	3rd Qu.: 0.73959
##	Max. : 20.00721	Max. : 15.59499	Max. : 23.74514	Max. : 12.01891
##	V12	V13	V14	V15
##	Min. : -18.6837	Min. : -5.79188	Min. : -19.2143	Min. : -4.49894
##	1st Qu.: -0.4056	1st Qu.: -0.64854	1st Qu.: -0.4256	1st Qu.: -0.58288
##	Median : 0.1400	Median : -0.01357	Median : 0.0506	Median : 0.04807
##	Mean : 0.0000	Mean : 0.00000	Mean : 0.0000	Mean : 0.00000
##	3rd Qu.: 0.6182	3rd Qu.: 0.66250	3rd Qu.: 0.4931	3rd Qu.: 0.64882
##	Max. : 7.8484	Max. : 7.12688	Max. : 10.5268	Max. : 8.87774
##	V16	V17	V18	
##	Min. : -14.12985	Min. : -25.16280	Min. : -9.498746	
##	1st Qu.: -0.46804	1st Qu.: -0.48375	1st Qu.: -0.498850	
##	Median : 0.06641	Median : -0.06568	Median : -0.003636	
##	Mean : 0.00000	Mean : 0.00000	Mean : 0.000000	
##	3rd Qu.: 0.52330	3rd Qu.: 0.39968	3rd Qu.: 0.500807	
##	Max. : 17.31511	Max. : 9.25353	Max. : 5.041069	
##	V19	V20	V21	
##	Min. : -7.213527	Min. : -54.49772	Min. : -34.83038	
##	1st Qu.: -0.456299	1st Qu.: -0.21172	1st Qu.: -0.22839	
##	Median : 0.003735	Median : -0.06248	Median : -0.02945	
##	Mean : 0.000000	Mean : 0.00000	Mean : 0.00000	

```
## 3rd Qu.: 0.458949 3rd Qu.: 0.13304 3rd Qu.: 0.18638
## Max. : 5.591971 Max. : 39.42090 Max. : 27.20284
## V22 V23 V24
## Min. :-10.933144 Min. :-44.80774 Min. :-2.83663
## 1st Qu.: -0.542350 1st Qu.: -0.16185 1st Qu.: -0.35459
## Median : 0.006782 Median : -0.01119 Median : 0.04098
## Mean : 0.000000 Mean : 0.00000 Mean : 0.00000
## 3rd Qu.: 0.528554 3rd Qu.: 0.14764 3rd Qu.: 0.43953
## Max. : 10.503090 Max. : 22.52841 Max. : 4.58455
## V25 V26 V27
## Min. :-10.29540 Min. :-2.60455 Min. :-22.565679
## 1st Qu.: -0.31715 1st Qu.: -0.32698 1st Qu.: -0.070839
## Median : 0.01659 Median : -0.05214 Median : 0.001342
## Mean : 0.00000 Mean : 0.00000 Mean : 0.000000
## 3rd Qu.: 0.35072 3rd Qu.: 0.24095 3rd Qu.: 0.091045
## Max. : 7.51959 Max. : 3.51735 Max. : 31.612198
## V28 Amount Class
## Min. :-15.43008 Min. : 0.00 Min. :0.000000
## 1st Qu.: -0.05296 1st Qu.: 5.60 1st Qu.:0.000000
## Median : 0.01124 Median : 22.00 Median :0.000000
## Mean : 0.00000 Mean : 88.35 Mean :0.001727
## 3rd Qu.: 0.07828 3rd Qu.: 77.17 3rd Qu.:0.000000
## Max. : 33.84781 Max. :25691.16 Max. :1.000000
```

3.4. Class Distribution

Finally, we will see what the breakdown of the data is, based on the ‘Class’ variable. This will also establish the imbalance that we are dealing with here.

```
table(fraud_data$Class)
```

```
##
##      0      1
## 284315  492
```

```
prop.table(table(fraud_data$Class))
```

```
##
##      0      1
## 0.998272514 0.001727486
```

As shown above, there is a clear imbalance, where less than 1% of the data is recorded as a fraudulent transaction.

4. Initial Notes on the Dataset

The dataset was loaded from the `creditcard.csv` file and was cleaned using the listwise deletion approach, which was through `na.omit()`. Following this step, there were no further missing values remaining in the dataset.

Inspecting the response variable, we can see how the data is highly imbalanced, with fraudulent transactions representing a small proportion of the entire data. This is an important feature of the problem, as this directly impacts the model fitting, prediction quality, and the choice of evaluation metrics.

The predictor variables are largely anonymised, limited the direct substantive interpretation of the individual features. However, the dataset remains useful for evaluating the model behaviour under extreme class imbalance.

Although the original dataset is structured as a fraud detection problem, this study extends the analysis into a survival framework through the later construction of a proxied time-to-event representation.

5. Data Preparation

In order to prepare for the section on the Exploratory Data Analysis (EDA), we want to ensure that the data has been generated and prepared appropriately, which is what we will focus on this section.

5.1. Create the Survival Variables

Firstly, we will work towards creating our respective survival variables, as this will set the scene to understand the structure of the data better.

```
fraud_data$time <- 1:nrow(fraud_data)      # Proxy time index
fraud_data$status <- fraud_data$Class      # Event indicator: 1 = fraud, 0 = non-fraud
```

5.2. Downsample Majority Class for Computational Efficiency

The original dataset is highly imbalanced and computationally large. To make this more robust, all fraud cases were retained, while a random sample of 50000 non-fraud cases were selected. This preserves the minority class of the substantive interest while reducing the runtime and memory burden of subsequent modelling work. This is what we call **downsampling**.

While this increases the fraud percentage by 0.8%, it also means that it stays well in line with the general percentage, wherein around 1% of transactions are recorded as a 'fraud'.

5.3 Train/Test Split

The dataset was partitioned into training and testing subsets using a stratified sampling approach. This ensures that the class distribution is preserved across both subsets, which is particularly important given the extreme imbalance in the dataset.

A 70–30 split was adopted, where 70% of the data is used for model training and the remaining 30% is reserved for out-of-sample evaluation. This approach allows for a robust assessment of model performance while ensuring sufficient data is available for training.

To further improve the stability of the results, this partitioning process is repeated multiple times in later sections. This repeated sampling framework helps reduce variance in performance estimates and provides a more reliable comparison between modelling approaches.

5.4. Create RSF Ready Train and Test Data

To enable the use of a survival-based modelling framework, the dataset was restructured to include the necessary components for Random Survival Forest (RSF) modelling. Specifically, the proxy time variable and event indicator (**status**) were used to define the survival object.

Since the **status** variable directly represents the fraud outcome, the original **Class** variable was removed from the modelling dataset to avoid duplication and potential leakage. This ensures that the model is trained using a consistent outcome representation.

The predictor variables were retained in their original form, with the exception of the time variable when required for specific preprocessing steps such as SMOTE. This results in a dataset that is compatible with RSF modelling while maintaining alignment with the original feature space.

This structured preparation ensures that both classification and survival-based models can be applied to a consistent dataset, enabling a fair comparison of their performance in later sections.

5.5. Benchmarking Sample

To support efficient experimentation and reduce computational overhead, a benchmarking sample of the training data is constructed. This subset allows for faster iteration during model development, particularly for computationally intensive procedures such as repeated resampling, SMOTE, and explainability methods.

A fixed-size random sample of the training dataset is taken, with a maximum of 50,000 observations. This ensures that the sample remains sufficiently large to preserve the underlying data structure while enabling more efficient computation.

The sampling process is performed using a fixed random seed to ensure reproducibility. Importantly, this benchmarking sample is used only for intermediate experimentation and validation, while final model evaluation is conducted on the full dataset to ensure robustness of results.

5.6. Notes on the RSF Data Construction

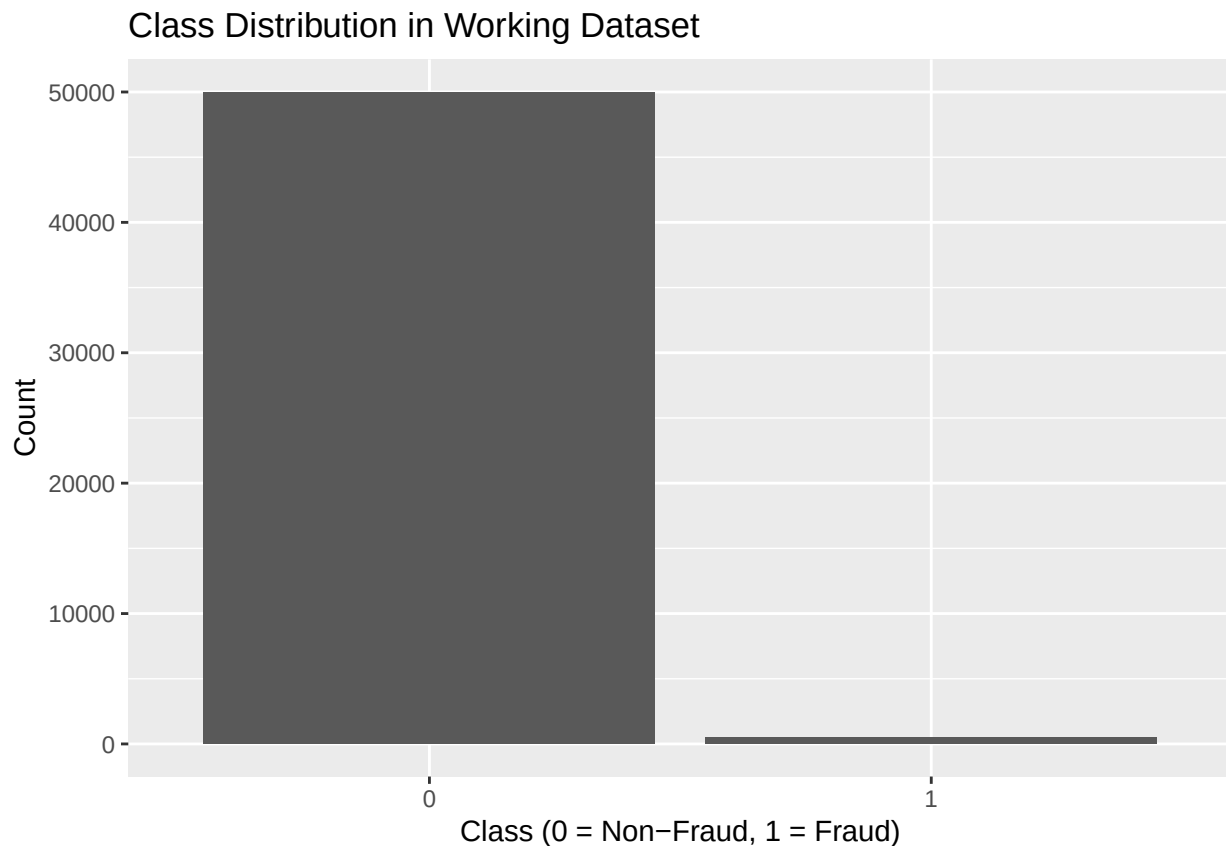
A proxy time index was created to enable a survival-style modelling framework. In this setup, `status` represents the fraud event indicator. Since `status` duplicates the outcome information contained in `Class`, the original `Class` variable was removed from the RSF-specific modelling data to avoid leakage and duplication.

Additionally, an optional benchmarking subset was created to support faster experimentation where required.

6. Exploratory Data Analysis

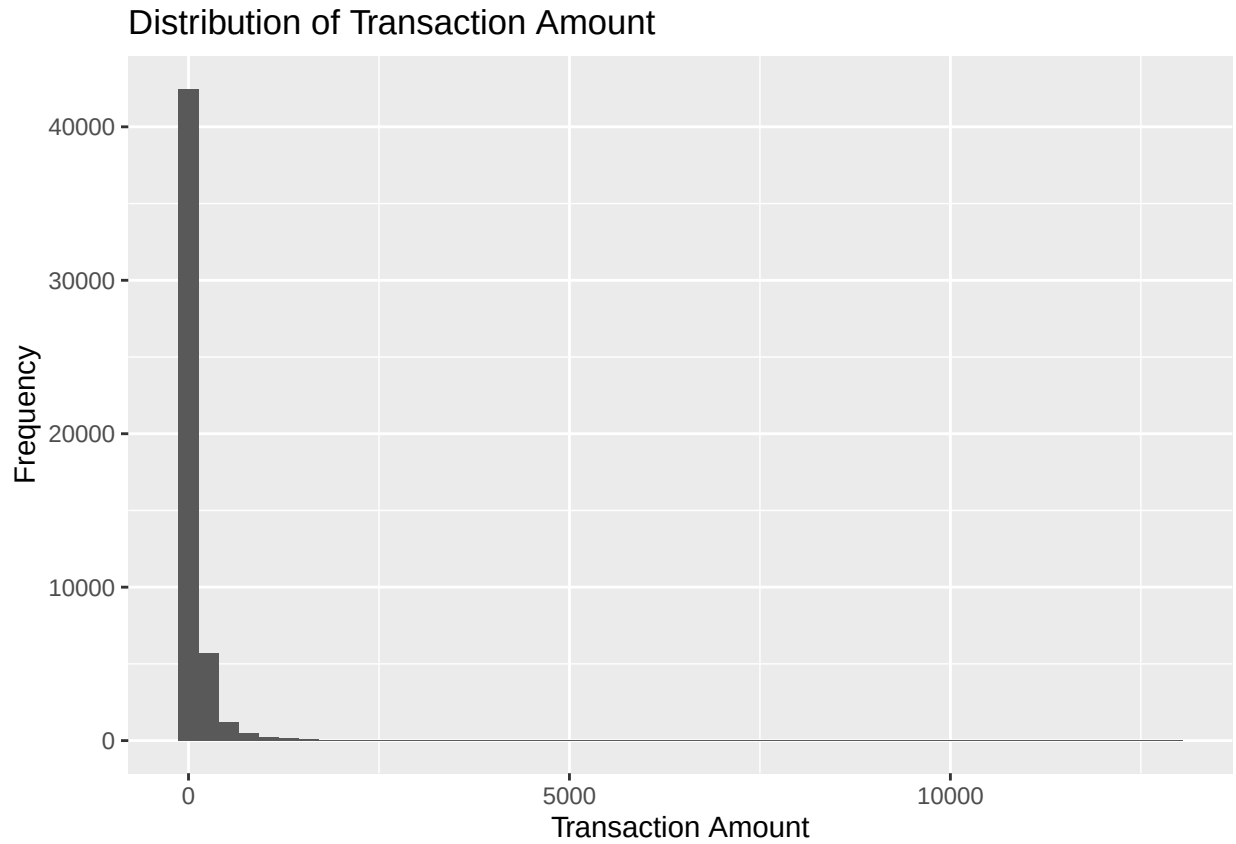
This stage will involve analysing the working dataset.

6.1. Class Distribution in the Working Dataset



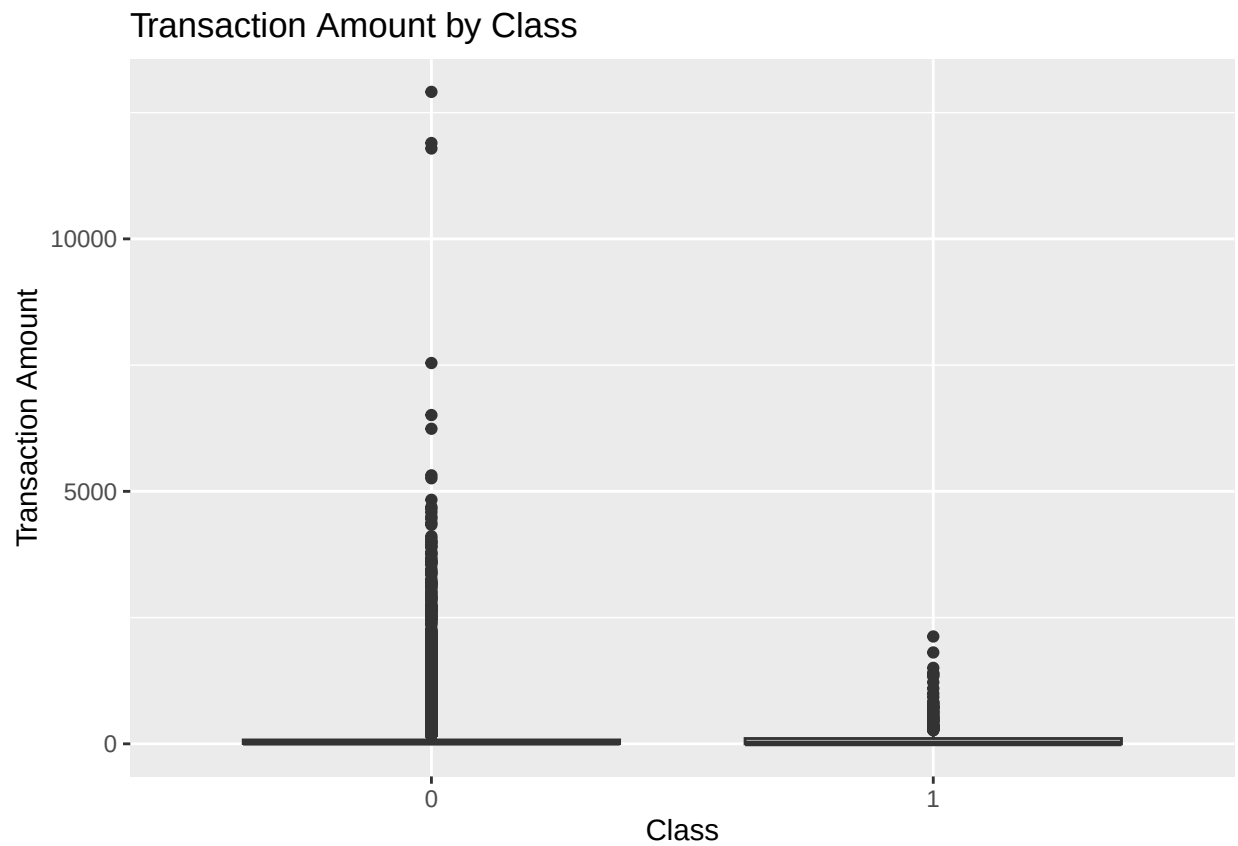
As previously established, there is an extreme class imbalance that exists in the dataset. The nature of this imbalance can be felt in the above visual.

6.2. Distribution of Transaction Amount



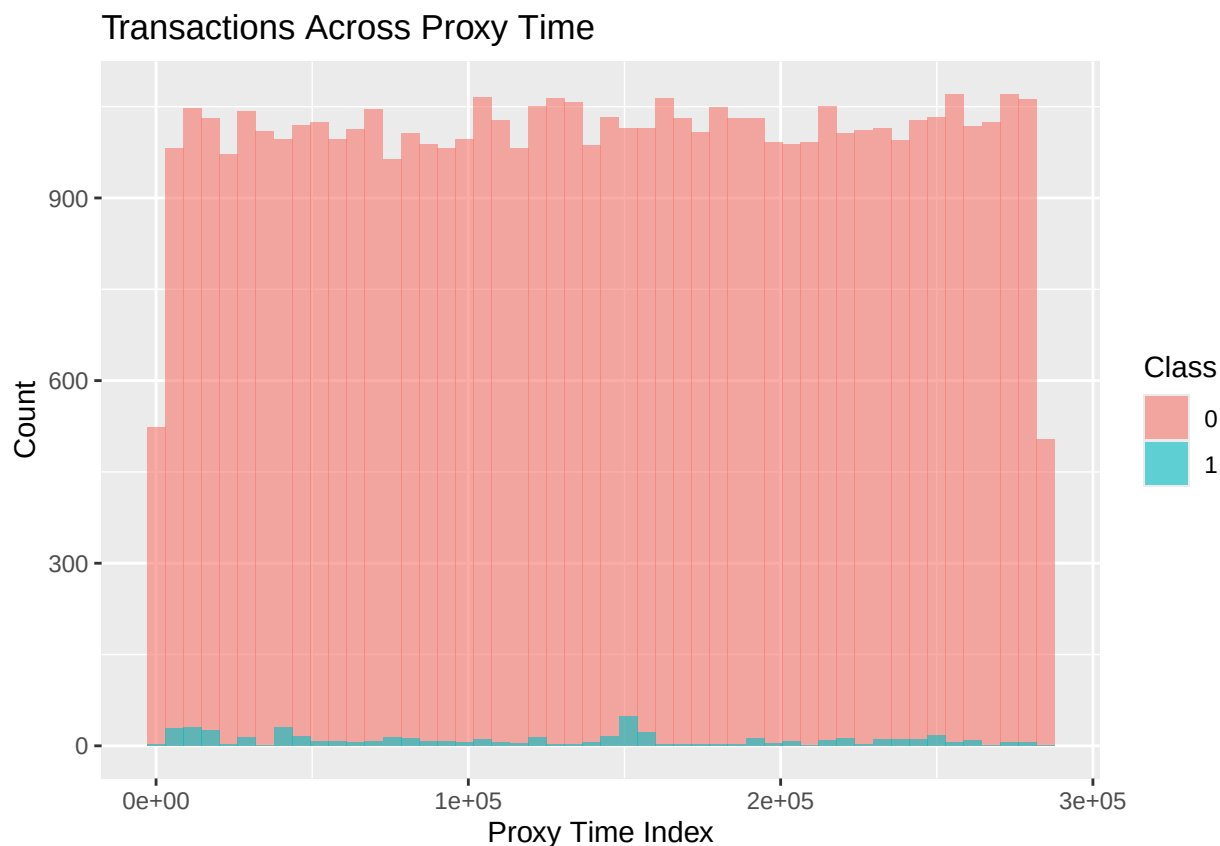
From the above, we can see that the transaction amounts are typically the small amounts. While this is true, it also shows how fraudulent transactions are predominantly low in value.

6.3. Transaction Amount by Class



The above shows us how the detected amounts are typically smaller, which as mentioned in the previous subsection, shows how the detected amounts are negligible amounts. This also means that people are most impacted when they least expect it.

6.4. Transactions across Proxy Time



6.5. EDA Commentary

- **Class Imbalance after Downsampling:** While the visualisation alone does not clearly convey the extent of the change, the class distribution shows that the fraud rate increases from approximately 0.17% in the original dataset to approximately 0.97% in the working subset. Although the dataset remains highly imbalanced, this represents a meaningful improvement that supports more stable model training while still reflecting realistic fraud prevalence levels.
- **Concentration of Fraud Cases in Proxy Time Index:** The distribution of fraud cases across the proxy time index appears relatively uniform, with no strong clustering in specific regions. However, it is important to note that the time variable is a constructed ordering index rather than a true temporal measure. As such, this observation does not imply that fraud is independent of time, but rather that no artificial ordering bias is introduced into the modelling framework.
- **Transaction Amount and Class:** The distribution of transaction amounts indicates that most transactions, including fraudulent ones, occur at relatively low monetary values. This suggests that fraud detection is not limited to high-value transactions and that models must be sensitive to subtle patterns in smaller transactions. However, the presence of higher-value outliers indicates that fraud is not exclusively associated with low amounts.
- **Suitability of later modelling choices:** The observed characteristics of the dataset support the use of both classification and survival-based modelling approaches. The persistent class imbalance motivates the inclusion of models that are robust to rare event detection, such as Random Forest and Random Survival Forest. Additionally, the absence of strong temporal clustering, combined with the constructed proxy time index, provides a basis for exploring a survival framework as a risk-ranking mechanism rather than a strict temporal prediction model. Together, these observations justify the

comparative modelling approach adopted in later sections.

7. Modelling Framework

7.1. Experimental Design

The modelling framework evaluates the performance of different approaches across two key dimensions:

1. Model type:
 - Logistic Regression (classification baseline)
 - Random Survival Forest (RSF)
2. Imbalance handling:
 - Without SMOTE
 - With SMOTE applied to the training data only

This results in four model configurations:

- Logistic Regression (No SMOTE)
- Logistic Regression (SMOTE)
- Random Survival Forest (No SMOTE)
- Random Survival Forest (SMOTE)

All models are evaluated using repeated stratified train-test splits to ensure robustness of results.

7.2. Logistic Regression (No SMOTE)

A baseline Logistic Regression model was fitted using the original imbalanced training data. No resampling techniques were applied in this setup, allowing the model to reflect standard classification behaviour under class imbalance.

7.3. Logistic Regression (SMOTE)

This model is used to assess whether synthetic balancing improves classification performance under extreme class imbalance. To address class imbalance, SMOTE was applied to the training data only. Synthetic minority samples were generated based on the feature space, while the test set remained untouched to preserve a realistic evaluation setting. The time variable was excluded from SMOTE generation to avoid introducing artificial temporal structure into the data.

7.4. Random Survival Forest (No SMOTE)

A Random Survival Forest (RSF) model was fitted using the survival representation of the data. The model leverages the proxy time index and event indicator to produce a risk-based prediction. No resampling was applied in this configuration, allowing assessment of RSF performance under naturally imbalanced conditions.

7.5. Random Survival Forest (SMOTE)

SMOTE was applied to the training data to address class imbalance prior to fitting the RSF model. Since SMOTE does not generate survival times, a resampling approach was used to assign time values to synthetic observations based on the empirical distribution of the original data.

Specifically: * SMOTE was applied to the feature space excluding the time variable

- Synthetic observations were generated for the minority class
- Time values were reassigned via sampling from the observed class-specific time distributions

This approach allows RSF to be trained on a balanced dataset, while acknowledging that the survival times of synthetic observations are approximations rather than true event times.

8. Results

This section presents the performance of all four model configurations:

- Logistic Regression (No SMOTE)
- Logistic Regression (SMOTE)
- Random Survival Forest (No SMOTE)
- Random Survival Forest (SMOTE)

All results are reported as the mean and standard deviation across 30 repeated stratified train-test splits.

8.1. Evaluation Metrics

The following metrics were used to evaluate model performance:

- Sensitivity (Recall)
- Precision
- AUC (ROC)
- AUPRC (Precision-Recall)

All metrics were averaged across repeated train-test splits.

8.2. Logistic Regression Results

Warning: package 'pander' was built under R version 4.5.3

Table 1: Table 1: Logistic Regression (No SMOTE) Performance Metrics

Metric	Mean	SD
Sensitivity	0.790425	0.0307794
Precision	0.942919	0.0181434
AUC	0.979472	0.00547542
AUPRC	0.866103	0.0206929

Table 2: Table 2: Logistic Regression (SMOTE) Performance Metrics

Metric	Mean	SD
Sensitivity	0.823068	0.0269521
Precision	0.908775	0.020082
AUC	0.97662	0.00803081
AUPRC	0.858829	0.0258725

Observations: The Logistic Regression model demonstrates moderate performance across all metrics. The introduction of SMOTE improves sensitivity, indicating an increased ability to detect fraudulent transactions. However, this improvement is accompanied by a reduction in precision, suggesting a higher rate of false positives.

8.3. Random Survival Forest Results

Table 3: Table 3: Random Survival Forest (No SMOTE) Performance Metrics

Metric	Mean	SD
Sensitivity	0.986859	0.0092969
Precision	0.0196173	0.00145209
AUC	0.973752	0.00751167
AUPRC	0.859151	0.0215466

Table 4: Table 4: Random Survival Forest (SMOTE) Performance Metrics

Metric	Mean	SD
Sensitivity	0.988682	0.0054473
Precision	0.0197157	0.00113344
AUC	0.97954	0.00548057
AUPRC	0.876069	0.0217121

Observations: The RSF model achieves consistently high sensitivity across both configurations, indicating strong capability in identifying fraudulent transactions. However, this is associated with lower precision, reflecting a trade-off where a larger number of transactions are flagged to minimise missed fraud cases.

8.4. Comparative Summary

Across all models, clear differences emerge in performance characteristics. Logistic Regression provides a stable baseline but is sensitive to class imbalance. The use of SMOTE improves detection performance, particularly in terms of sensitivity.

In contrast, the RSF model demonstrates superior sensitivity, suggesting that a risk-based modelling approach may be more effective for identifying fraudulent transactions. However, this comes at the cost of lower precision, highlighting the trade-off between detection and false positives.

##	Metric	Mean	SD	Model	SMOTE
## 1	Sensitivity	0.79042501	0.030779364	LR	No
## 2	Precision	0.94291859	0.018143384	LR	No
## 3	AUC	0.97947165	0.005475422	LR	No
## 4	AUPRC	0.86610285	0.020692857	LR	No
## 5	Sensitivity	0.82306782	0.026952091	LR	Yes
## 6	Precision	0.90877515	0.020081986	LR	Yes
## 7	AUC	0.97661975	0.008030814	LR	Yes
## 8	AUPRC	0.85882935	0.025872521	LR	Yes
## 9	Sensitivity	0.98685859	0.009296898	RSF	No
## 10	Precision	0.01961728	0.001452092	RSF	No
## 11	AUC	0.97375208	0.007511668	RSF	No
## 12	AUPRC	0.85915149	0.021546637	RSF	No
## 13	Sensitivity	0.98868165	0.005447300	RSF	Yes
## 14	Precision	0.01971572	0.001133438	RSF	Yes
## 15	AUC	0.97954046	0.005480574	RSF	Yes
## 16	AUPRC	0.87606891	0.021712065	RSF	Yes

The results of this study highlight the following key findings:

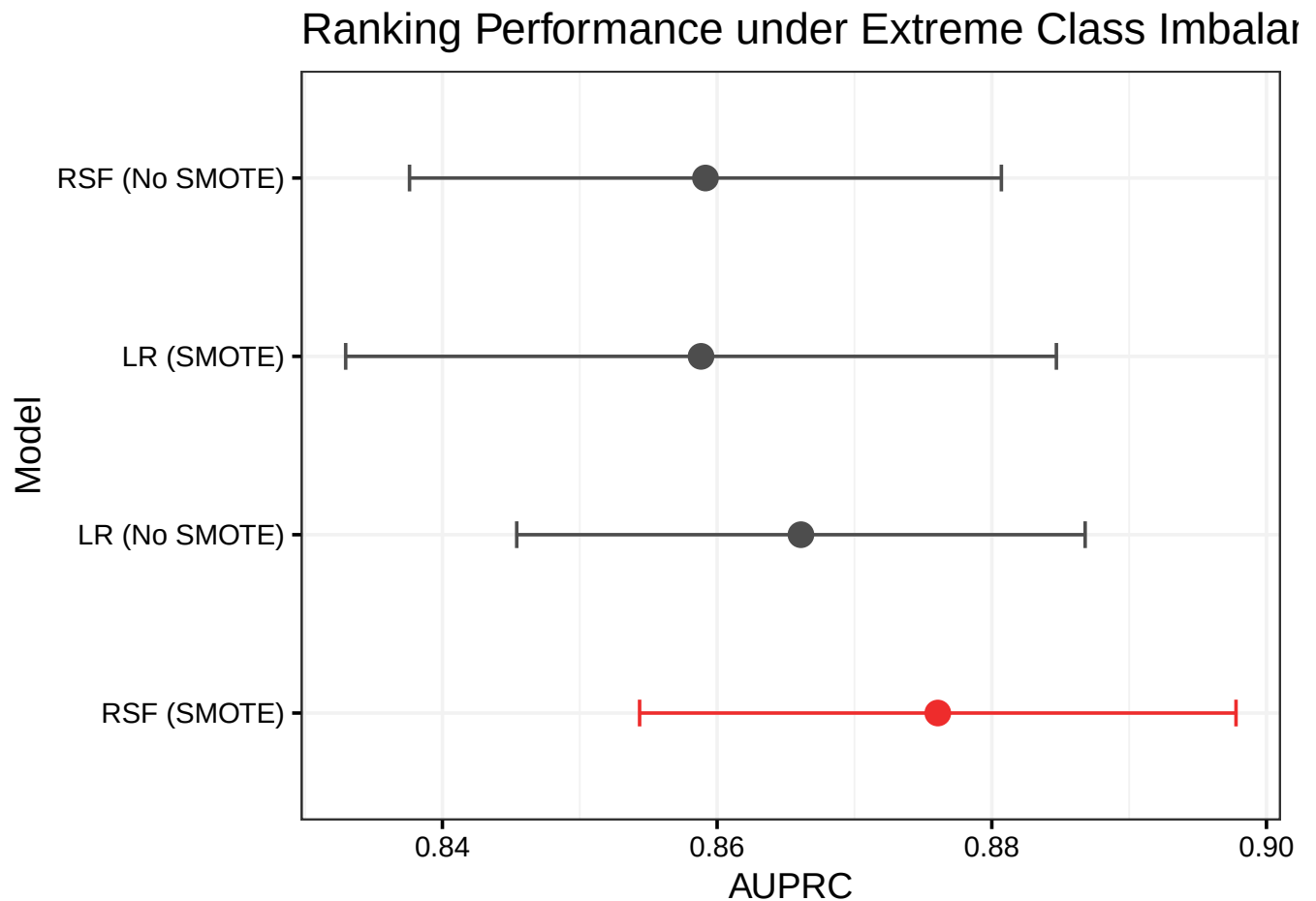
- SMOTE improves sensitivity across all models, confirming its effectiveness in addressing class imbalance.
- Logistic Regression provides a stable baseline but is sensitive to imbalance effects.

- The RSF model achieves consistently high sensitivity, indicating strong fraud detection capability.
- The combination of RSF and SMOTE yields strong overall performance, particularly in terms of ranking and recall.
- A trade-off between sensitivity and precision is observed across all models.

8.5. Visual Comparison

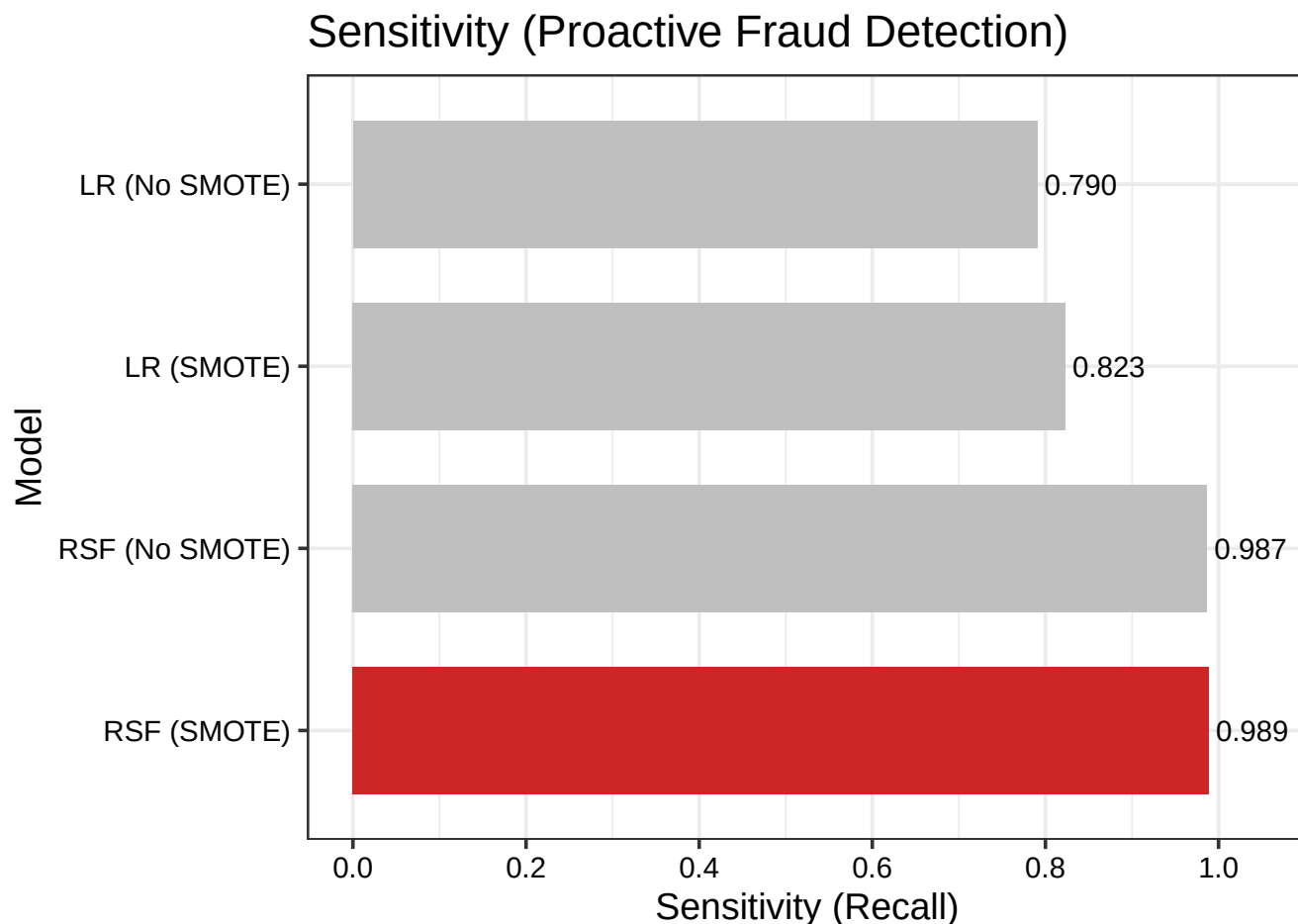
To compare model performance more clearly, two summary plots are presented below. The first focuses on AUPRC, which is especially informative under severe class imbalance. The second focuses on sensitivity, reflecting the ability of each model to correctly identify fraudulent transactions.

8.5.1. AUPRC Comparison



The AUPRC comparative highlights the ranking performance of each model under extreme class imbalance. This metric is particularly important in fraud detection because it places emphasis on the model's ability to prioritise true fraud cases without being overly influenced by the dominant non-fraud class. The results suggest that the RSF model with SMOTE achieves the strongest ranking performance overall.

8.5.2. Sensitivity Comparison Below is the outline of the Sensitivity plot.



The sensitivity plot shows that the RSF-based approaches achieve the strongest recall, indicating that they are more effective at identifying fraudulent cases. This is important in fraud detection settings where missing a fraud event may be more costly than raising additional false positives. By contrast, the Logistic Regression models show lower sensitivity, particularly without SMOTE, suggesting that class imbalance has a stronger negative effect on standard classification performance.

9. Explainability

9.1 Why Explainability Matters in Fraud Detection

In fraud detection, model performance alone is not sufficient. Banking environments require models that are not only effective, but also interpretable. Explanations are important for several reasons: they help analysts understand why a transaction has been flagged, support internal validation and governance processes, and improve confidence in model-assisted decision-making.

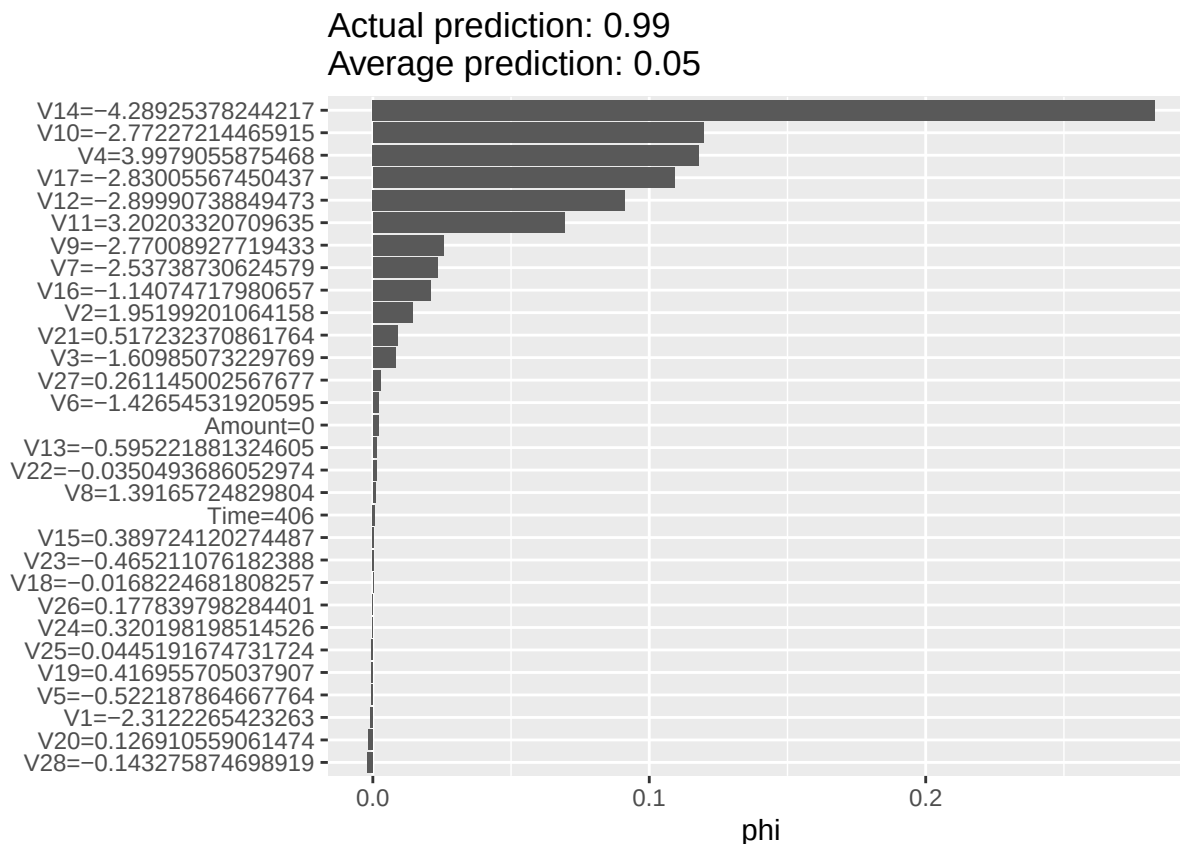
This is particularly relevant in the present study, where the Random Survival Forest produces a risk-based score rather than a simple class probability. Explainability methods therefore help connect the model output to transaction-level reasoning.

9.2 Global Explainability

Global explainability provides an overall view of which predictors contribute most strongly to the model's behaviour across the dataset. This is useful for identifying broad fraud-related patterns and determining whether the model is relying on a concentrated set of meaningful features.

9.3 Local Explainability using SHAP

While global importance identifies which variables matter overall, SHAP provides a local explanation for a single transaction. This allows the contribution of each feature to the RSF risk score to be decomposed at the transaction level. In this study, SHAP is applied to the RSF model with SMOTE in order to examine how individual predictors increase or decrease the estimated fraud risk for a selected observation.

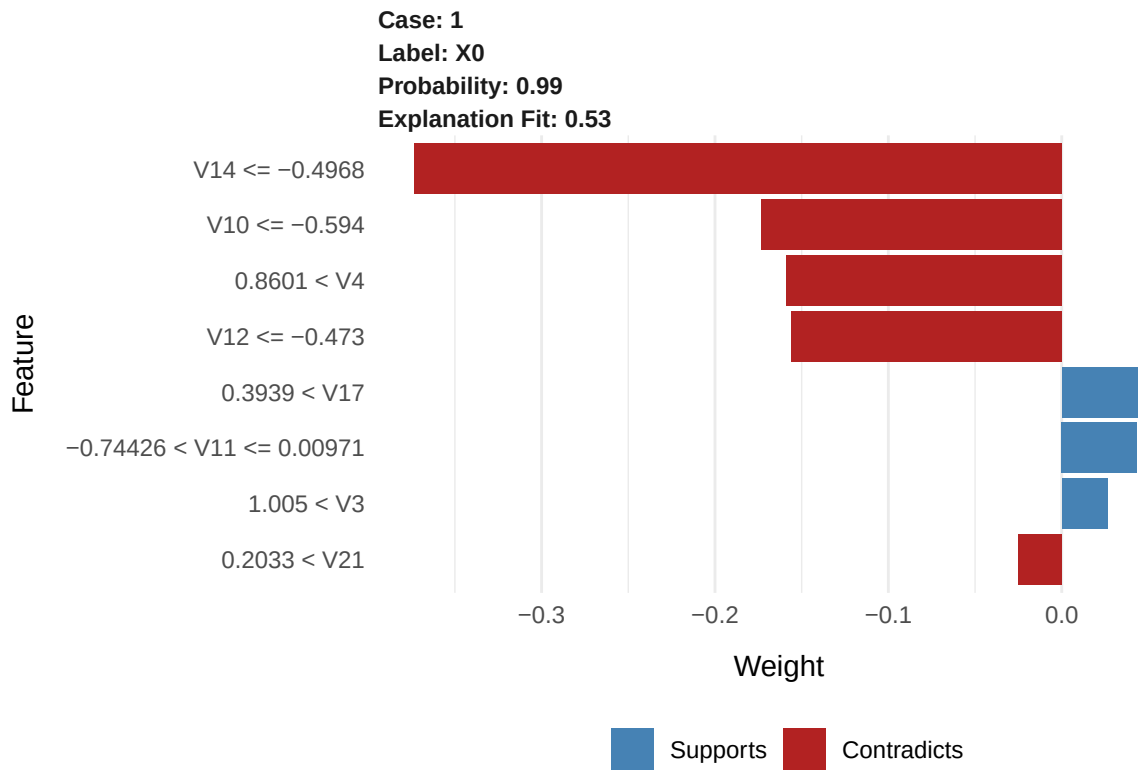


The SHAP explanation shows how the selected transaction's risk score is built up from individual feature contributions. Positive contributions move the transaction toward higher estimated fraud risk, while negative contributions move it toward lower estimated risk.

From a banking perspective, this is useful because it mirrors the logic of case investigation. Rather than receiving only a risk score, an analyst can see which characteristics of the transaction most strongly influenced the model's assessment.

9.4 Local Explainability using LIME

Local Interpretable Model-agnostic Explanations (LIME) provides a second local explanation framework by approximating the model decision around a selected transaction with a simpler interpretable representation. In this study, LIME is used to complement SHAP by showing which features most strongly support the local fraud classification for an individual case.



LIME provides a complementary local explanation framework by approximating the behaviour of the RSF model in the neighbourhood of a selected transaction using a simpler, interpretable model. This allows the contribution of a small number of features to be expressed in a linear and human-readable form.

The LIME output shows both positive and negative contributions. Features with positive weights increase the likelihood of the transaction being classified as fraudulent, while those with negative weights reduce it. This provides a directional explanation of the decision, rather than simply identifying important variables.

While SHAP provides a theoretically grounded decomposition of the prediction, LIME offers a simpler and more interpretable local approximation. The two methods therefore complement each other, with SHAP emphasising consistency and LIME emphasising interpretability.

From a banking perspective, this type of explanation aligns closely with how fraud analysts review cases. This directional interpretation allows analysts to understand not only which features matter, but how they influence the model's decision. This supports faster investigation and more informed decision-making.

However, it is important to note that LIME explanations are based on a local approximation of the model and may not fully capture complex interactions present in the RSF model. As a result, explanations should be interpreted as indicative rather than exact.

9.5 Banking Relevance of Explainability

The explainability results strengthen the practical relevance of the modelling framework. In a banking environment, fraud detection systems are not used in isolation - they support fraud operations teams who must review alerts, prioritise investigations, and justify intervention decisions.

Global explainability helps identify the broad drivers of suspicious behaviour across the portfolio, while local explanation methods such as SHAP and LIME provide transaction-level insight into why a case was flagged. This is particularly valuable for analyst workflows, model governance, and customer-facing situations where a

flagged transaction may require review or escalation.

More broadly, the inclusion of explainability addresses one of the key limitations of black-box fraud models. Rather than producing only a ranked risk score, the framework developed in this study also provides a mechanism for understanding how that score was formed.

10. Discussion

10.1. Model Performance Comparison

The results show that both model choice and imbalance handling significantly influence fraud detection performance. Logistic regression provides a strong baseline, but is sensitive to class imbalance. The introduction of SMOTE however, improves sensitivity, though it is at the expense of precision.

In contrast, the Random Survival Forest (RSF) demonstrates consistently strong sensitivity, indicating a survival-based risk framework may be more effective in identifying fraudulent transactions.

10.2. Classification vs Survival

A key distinction in this study lies in probability-based classification and risk-based survival modelling. Classification models estimate the probability that a transaction is fraudulent, whereas the RSF model estimates the relative risk of a fraud occurrence. This allows transactions to be ranked, rather than simply being classified.

This ranking-based approach aligns more closely with real-world fraud detection systems, where the prioritisation of high-risk transactions are often more important than binary classification.

10.3. Impact of SMOTE

SMOTE improves the sensitivity across both modelling approaches, confirming its effectiveness in handling class imbalance. However, its use in survival modelling introduces additional considerations, as synthetic observations do not naturally include time-to-event information relevant to modelling. As a result, this highlights a trade-off between improved detection performance and methodological consistency.

10.4 Role of Explainability

The inclusion of SHAP and LIME strengthens the interpretability of the modelling framework. While the RSF introduces a more complex, risk-based approach, the explainability techniques allow this complexity to be translated into meaningful transaction-level insights.

This is particularly important in fraud detection, since model outputs must be interpretable for operational and governance processes.

11. Banking Implications

11.1. Fraud Detection Operations

In practical banking environments, fraud detection systems must balance detection accuracy with operational efficiency. High sensitivity is particularly valuable, since missed fraudulent transactions have significant financial and reputational consequences to both financial institutions and its customers.

11.2. Risk Prioritisation

The RSF model enables transactions to be ranked by risk, allowing fraud teams to prioritise investigations. This is especially important in high-volume settings where this is not feasible to review all transactions manually. Hence, this ranking-based approach aligns closely with real-world fraud detection workflows.

11.3. Explainability and Trust

Explainability techniques such as SHAP and LIME provide transparency into model decisions, supporting analyst understanding and regulatory compliance. This helps build trust in model-assisted decision-making and supports effective deployment in banking environments.

12. Limitations

This study had several limitations, namely:

- The time variable used in the survival framework is a proxy, rather than a true temporal measure.
- SMOTE does not generate survival times, requiring synthetic assignment of the time values.
- Downsampling reduces the level of representation that the original dataset contains.
- RSF classification requires a threshold, which is chosen through experimentation.
- Explainability results are demonstrated on a limited number of observations.

13. Conclusion

This study compared classification and survival-based modelling approaches to fraud detection under extreme class imbalance. The results themselves show that: - SMOTE improves sensitivity across models - RSF provides a strong risk-based alternative to classification - RSF combined with SMOTE achieves strong detection performance

More importantly, the survival-based framework introduces a ranking perspective that aligns more closely with real-world fraud detection systems. The inclusion of explainability techniques further enhances the practical applicability of the approach.

Future work should focus on improving temporal modelling and extending explainability to broader transaction sets.